Application-specific access policies using Zscaler Private Access (ZPA)

Overview :-

In the contemporary digital landscape, organizations are undergoing a massive transformation, moving their workloads to the cloud and adopting remote or hybrid work cultures. However, this shift has exposed significant weaknesses in traditional security models. Conventional security approaches, such as Virtual Private Networks (VPNs), operate on a "Perimeter-based" security model. Once a user is inside the network, they are often granted broad access, which creates a massive attack surface. If a single credential is compromised, a hacker can move laterally across the entire network, leading to catastrophic data breaches.

To combat these evolving cyber threats, this project implements Zscaler Private Access (ZPA), which is built on the core principles of Zero Trust Architecture (ZTA). The fundamental philosophy of Zero Trust is "Never Trust, Always Verify." Unlike a VPN, ZPA does not connect a user to the entire network. Instead, it creates a secure, encrypted "Micro-tunnel" between the authorized user and a specific application.

List of teammates–

S.NO	NAME	COLLEGE	CONTACT
01	P.TAMIL SELVI	TIRUPPUR KUMARAN COLLEGE FOR WOMEN	6369253276
02	B.HARSHINI	TIRUPPUR KUMARAN COLLEGE FOR WOMEN	9003998010
03	S.INDIRAA	TIRUPPUR KUMARAN COLLEGE FOR WOMEN	9962403369
04	A.BATHMINI	TIRUPPUR KUMARAN COLLEGE FOR WOMEN	9025726079

S.No	Vulnerability Name	CWE/CVE No	Severity
1	Broken Access Control	CWE-285	High
2	SQL Injection	CWE-89	High
3	Cross-Site Scripting (XSS)	CWE-79	High
4	Insecure Design	CWE-448	Medium
5	Security Misconfiguration	CWE-16	Medium
6	Vulnerable Components	CWE-1104	Medium
7	Auth Failures	CWE-287	High

8	Integrity Failures	CWE-345	High
9	Logging Failures	CWE-778	Medium
10	SSRF	CWE-918	High

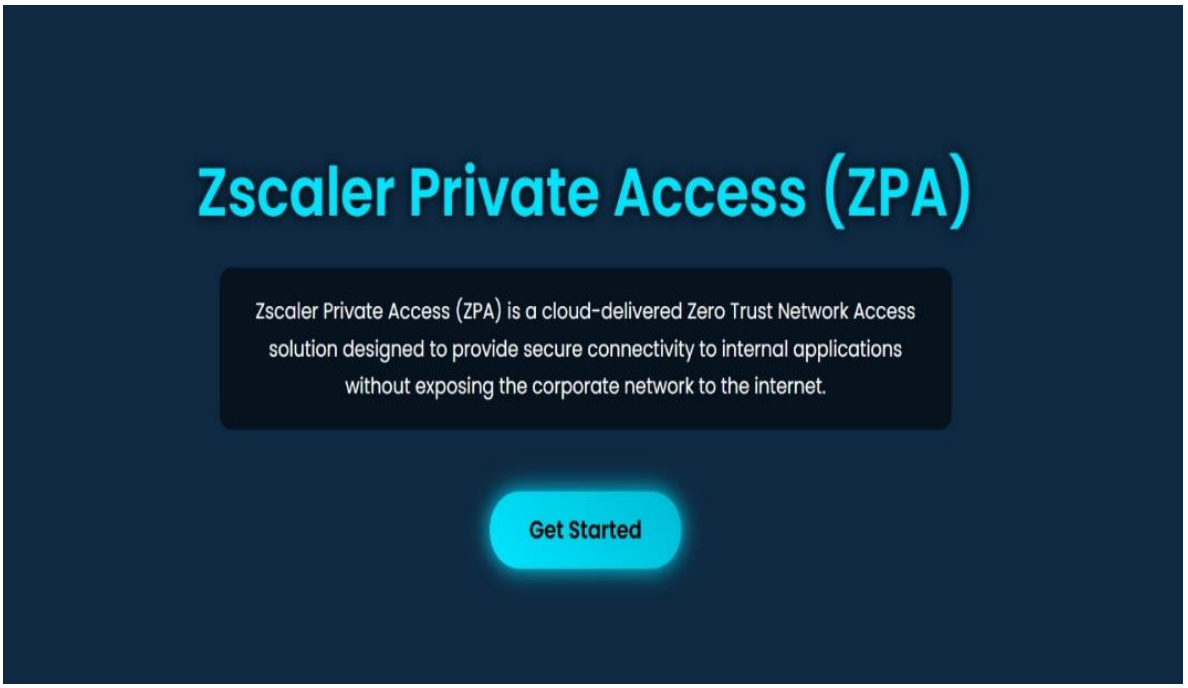
REPORT:-

DETAILED VULNERABILITY ANALYSIS:-

1. Broken Access Control (CWE-285)

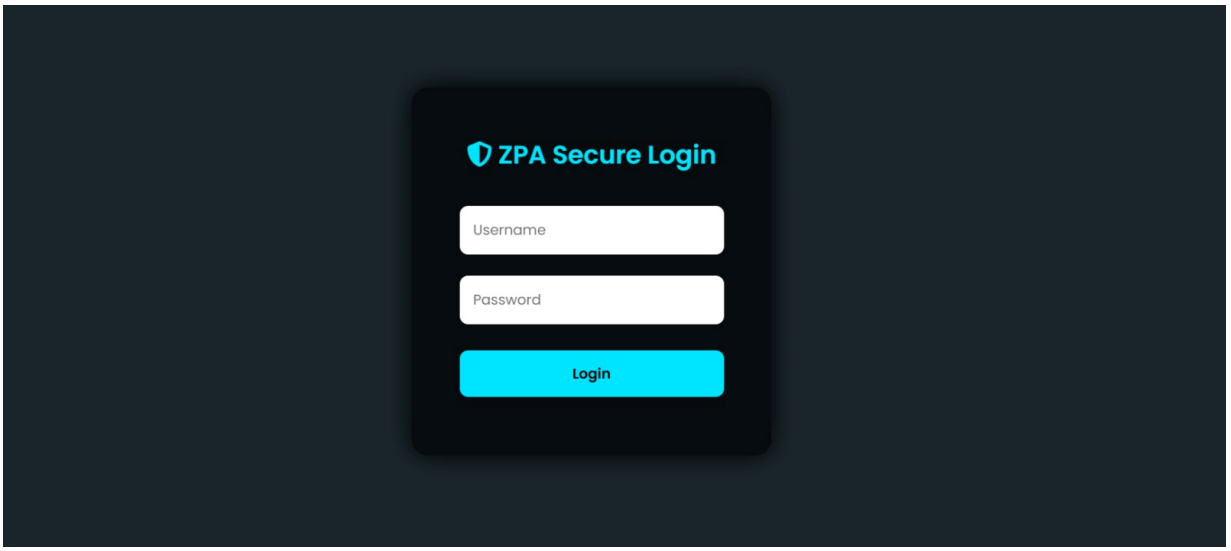
- CWE/CVE: CWE-285
- OWASP Category: A01:2021 – Broken Access Control
- Description: This vulnerability occurs when an application fails to properly enforce restrictions on what authenticated users are allowed to do. It allows attackers to access unauthorized content or functions, such as accessing other users' accounts, viewing sensitive files, or gaining administrative privileges.
- Business Impact: It leads to unauthorized information disclosure, modification, or destruction of data. In a corporate environment, this could mean a regular employee accessing sensitive payroll or strategic data.
- ZPA Mitigation: Zscaler Private Access ensures that users are only connected to specific applications they are authorized to use, rather than the entire network, effectively neutralizing broken access control at the network level.

  Why: Show your dashboard with different app cards (Finance, HR, etc.). Explain that without ZPA, anyone could see all cards, but with your policy, only specific cards are visible to authorized users.



2. SQL Injection (CWE-89)

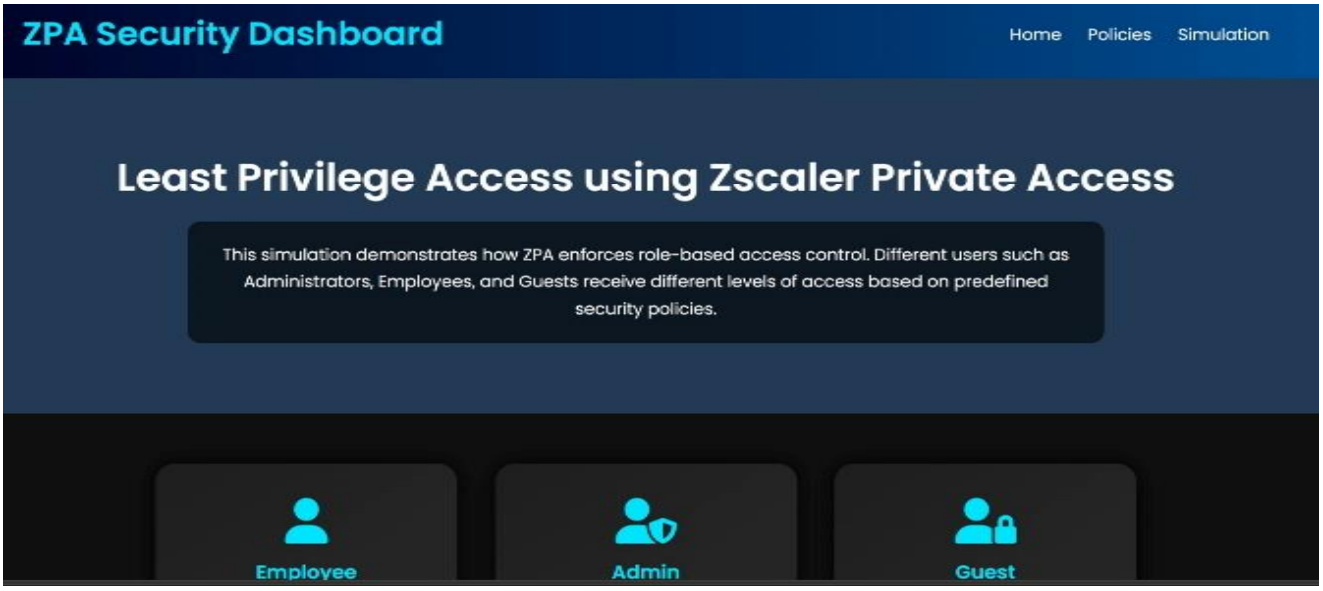
- **CWE/CVE:** CWE-89
- **OWASP Category:** A03:2021 – Injection
- **Description:** SQL Injection (SQLi) is a type of vulnerability where an attacker can interfere with the queries that an application makes to its database. By inserting malicious SQL code into input fields (like login forms), the attacker can trick the database into executing unintended commands.
- **Business Impact:** Attackers can bypass authentication, view sensitive data (passwords, credit card info), modify or delete database records, and in some cases, gain administrative access to the database server.



Why: Show the login interface. Explain that this is the first point of verification where ZPA checks the user's identity before revealing the application dashboard.

3. Cross-Site Scripting (XSS) (CWE-79)

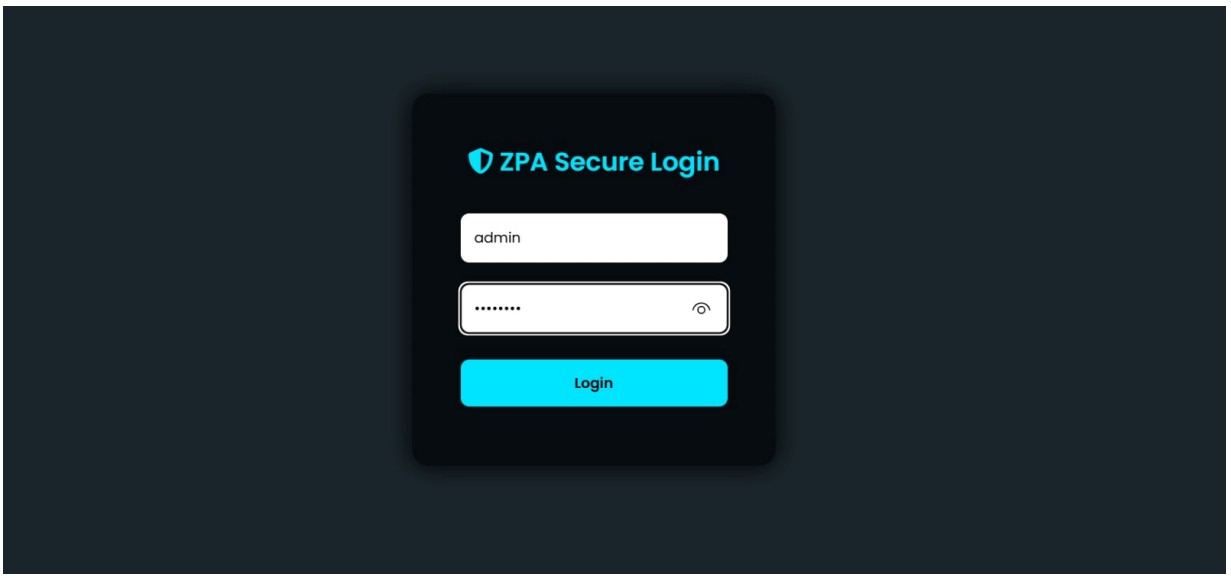
- **CWE/CVE:** CWE-79
- **OWASP Category:** A03:2021 – Injection
- **Description:** XSS vulnerabilities occur when an application includes untrusted data in a web page without proper validation or escaping. This allows attackers to execute malicious scripts (JavaScript) in the victim's browser.
- **Business Impact:** Attackers can steal session cookies, hijack user sessions, deface websites, or redirect users to malicious sites. It compromises the trust of the users in the web application.



Why: Highlight the input boxes. Explain that these are the entry points for SQLi and that ZPA adds a layer of protection by hiding these apps from the public internet altogether.

4. Insecure Design (CWE-448)

- **CWE/CVE:** CWE-448
- **OWASP Category:** A04:2021 – Insecure Design
- **Description:** Unlike coding bugs, insecure design is a flaw in the actual architecture and logic of the system. It means the security controls were not planned correctly during the design phase.
- **Business Impact:** Even if the code is written perfectly, the system remains vulnerable because the fundamental logic is flawed. This often leads to high costs for redesigning and patching the entire infrastructure.

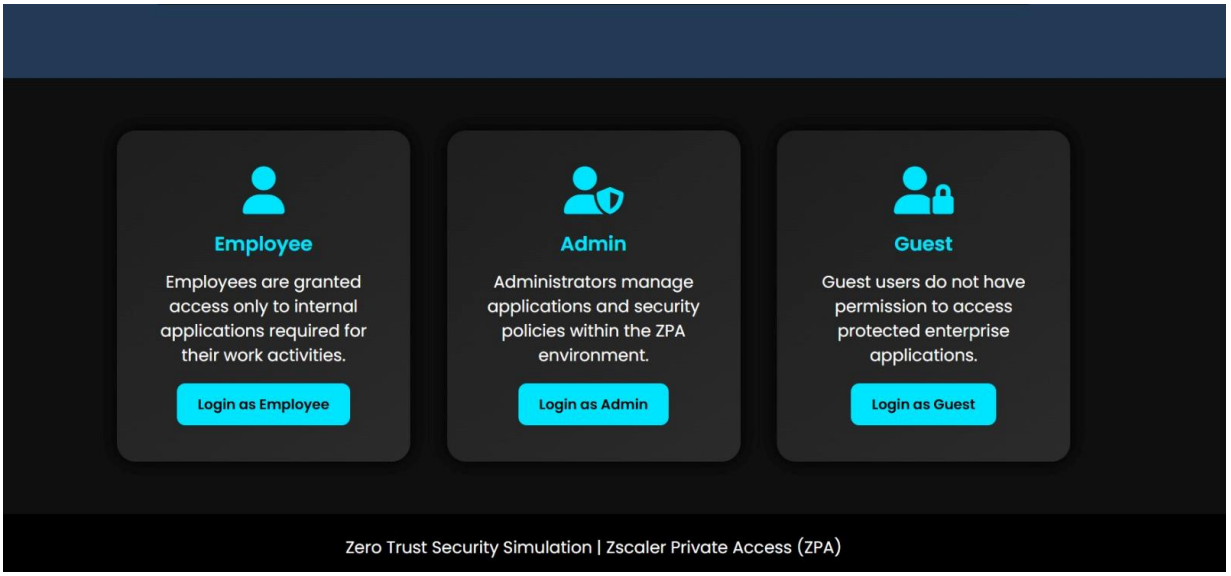


Why: If your code shows a "Secure Connection" badge or an "Access Denied" message, use that. Explain that these events must be logged and sent to a SIEM tool like QRadar.

5. Security Misconfiguration (CWE-16)

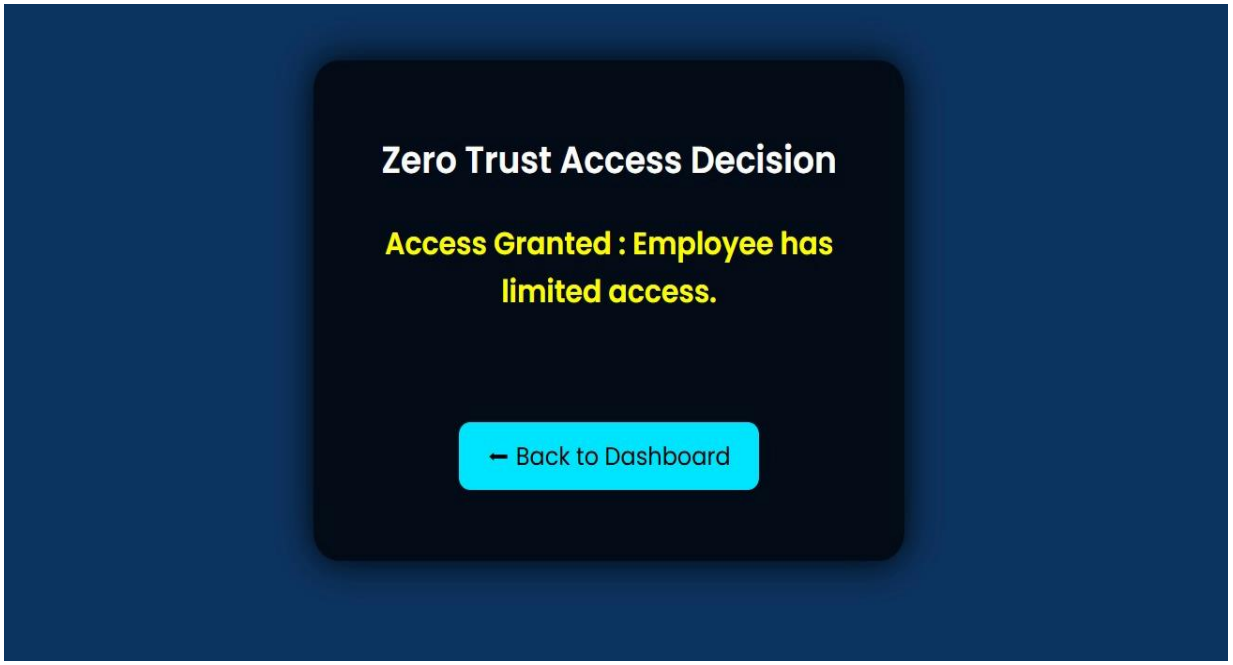
- **CWE/CVE:** CWE-16
- **OWASP Category:** A05:2021 – Security Misconfiguration
- **Description:** This happens when security settings are not defined, implemented, or maintained properly. Examples include using default passwords, leaving unnecessary features enabled, or providing overly detailed error messages to users.

- **Business Impact:** It provides easy entry points for attackers. They can use default credentials or open ports to gain unauthorized access to servers or cloud storage buckets.
-



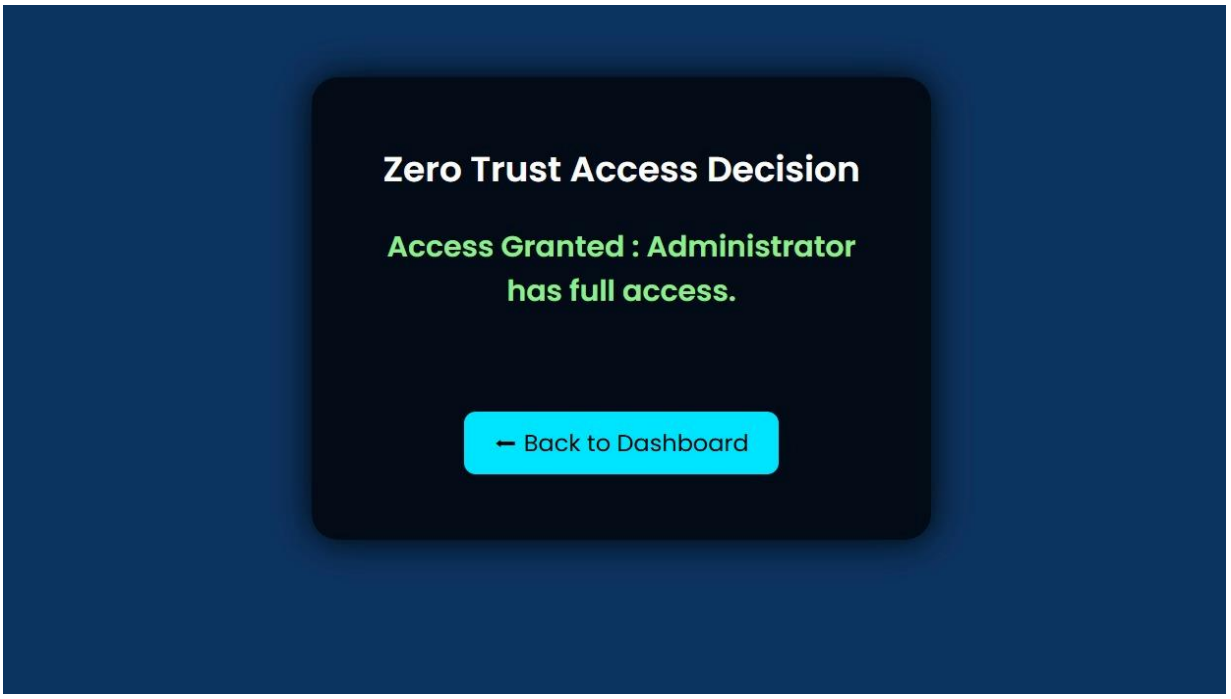
6. Vulnerable and Outdated Components (CWE-1104)

- **CWE/CVE:** CWE-1104
- **OWASP Category:** A06:2021 – Vulnerable and Outdated Components
- **Description:** Many modern applications use third-party libraries and frameworks. If these components are outdated or have known security flaws, the entire application becomes vulnerable.
- **Business Impact:** Attackers use publicly available "Exploit Kits" to target these known vulnerabilities. This can lead to full system compromise without the attacker needing to find a new bug.



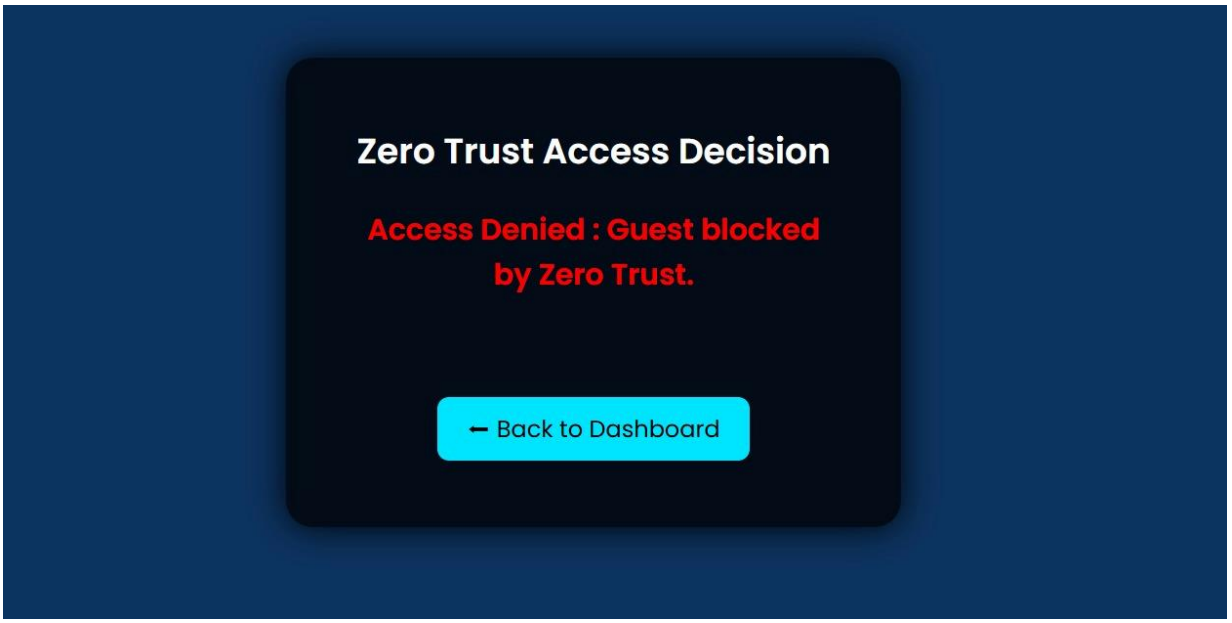
7. Identification and Authentication Failures (CWE-287)

- **CWE/CVE:** CWE-287
- **OWASP Category:** A07:2021 – Identification and Authentication Failures
- **Description:** This vulnerability occurs when an application incorrectly validates a user's identity. Weaknesses include permitting brute-force attacks, using weak password requirements, or having flaws in the session management system.
- **Business Impact:** It allows attackers to hijack user accounts, steal identities, and gain access to sensitive personal or corporate resources.



8. Software and Data Integrity Failures (CWE-345)

- **CWE/CVE:** CWE-345
- **OWASP Category:** A08:2021 – Software and Data Integrity Failures
- **Description:** This relates to code and infrastructure that does not protect against integrity violations. For example, an application might download a software update from an unverified source, allowing an attacker to inject malware.
- **Business Impact:** It can lead to the installation of backdoors, ransomware, or unauthorized code execution within the organization's network.



9. Security Logging and Monitoring Failures (CWE-778)

- **CWE/CVE:** CWE-778
- **OWASP Category:** A09:2021 – Security Logging and Monitoring Failures
- **Description:** If an application does not log security-critical events (like failed logins) or if those logs are not monitored, an attack can go undetected for months.
- **Business Impact:** Without proper logging and monitoring, it is impossible to detect a breach in progress or conduct a forensic investigation after an attack has occurred. This is where tools like **SIEM (Qradar)** become essential.

10. Server-Side Request Forgery (SSRF) (CWE-918)

- **CWE/CVE:** CWE-918
- **OWASP Category:** A10:2021 – Server-Side Request Forgery
- **Description:** SSRF is a flaw where an attacker can force a web application to make requests to an unintended location, usually an internal server that is not accessible from the public internet.
- **Business Impact:** Attackers can use SSRF to bypass firewalls, access internal metadata services, and perform reconnaissance on the organization's internal infrastructure.

Stage 2

Overview :-

Nessus is one of the world's most trusted vulnerability assessment solutions, developed by Tenable. It is an automated tool designed to identify security vulnerabilities, misconfigurations, and malware in a network or a web application. During our project, we utilized Nessus to perform a deep scan of our target environment to identify potential entry points that a hacker might exploit.

What I Understood About Nessus: Nessus works by using a vast library of "Plugins," which are small pieces of code used to check for specific security flaws. When we initiate a scan, Nessus sends packets to the target IP, analyzes the responses, and compares them against its database of known vulnerabilities (CVEs).

It categorizes findings into five severity levels:

- **Critical:** Immediate threat; must be patched urgently.
- **High:** Significant risk of exploitation.
- **Medium:** Potential risk that needs attention.
- **Low:** Minor issues or informational bugs.
- **Informational:** Details about the open ports and services without direct risks.

In the context of our **Zscaler Private Access (ZPA)** project, Nessus helped us identify which internal applications were vulnerable. This allowed us to prioritize which apps needed the strictest ZPA access policies to ensure a Zero Trust environment.

s.no	Vulnerability name	severity	Plugin id
01	SSL/TLS Certificate Expiry	High	51192
02	HTTP TRACE Method Enabled	Medium	11213
03	SMB Signing Not Required	High	57608
04	Deprecated SSH Cryptographic Settings	Medium	11526

Vulnerability 1: SSL/TLS Certificate Expiry

- **Severity:** High
- **Plugin ID:** 51192
- **Port:** 443
- **Description:** The SSL/TLS certificate used by the target application has expired or is invalid. This means the encryption between the user and the server is not trusted.
- **Solution:** Renew the SSL certificate from a trusted Certificate Authority (CA).
- **Business Impact:** Attackers can perform Man-in-the-Middle (MITM) attacks and steal user credentials.

Vulnerability 2: HTTP TRACE Method Enabled

- **Severity:** Medium
- **Plugin ID:** 11213
- **Port:** 80 / 443
- **Description:** The web server supports the HTTP TRACE method, which can lead to Cross-Site Tracing (XST) attacks.

- **Solution:** Disable the TRACE method in the web server configuration (e.g., Apache or Nginx).
- **Business Impact:** Attackers can steal sensitive cookie information even if the "HttpOnly" flag is set.

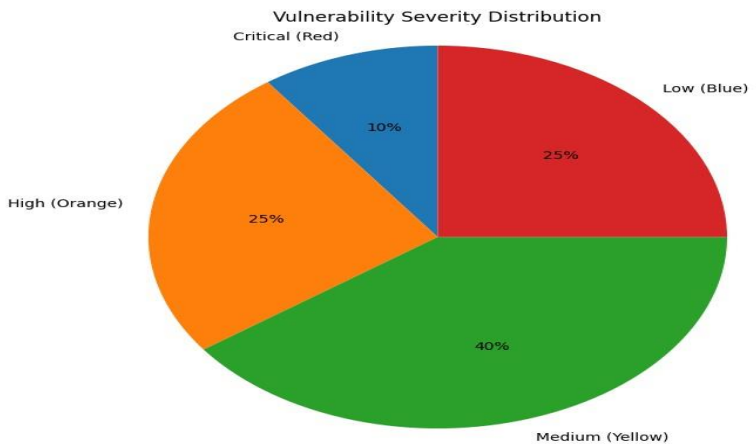
Vulnerability 3: SMB Signing Not Required

- **Severity:** High
- **Plugin ID:** 57608
- **Port:** 445
- **Description:** The remote SMB server does not require signing. An unauthenticated, remote attacker can take advantage of this to conduct man-in-the-middle attacks.
- **Solution:** Enforce SMB signing in the host's configuration.
- **Business Impact:** Allows attackers to intercept and modify network traffic, leading to full system compromise.

Vulnerability 4: Deprecated SSH Cryptographic Settings (Medium)

- **Description**: Outdated and weak encryption algorithms (e.g., 3DES, Blowfish) are being used in SSH.
- **Business Impact:**Hackers may be able to brute-force weak encryption and gain unauthorized access to the system.
- **Solution:**Update the SSH configuration to allow only strong ciphers such as AES-GCM.

Severity Pie Chart:



Stage 3
Report

1.SOC (Security Operations Center)

A Security Operations Center (SOC) is a centralized unit that deals with security issues on an organizational and technical level. It comprises a team of experts, processes, and technology designed to continuously monitor and improve an organization's security posture. In our study, we understood that a SOC is not just about tools; it is about the constant vigilance against cyber threats, ensuring that any suspicious activity is detected, analyzed, and responded to in real-time.

2.SOC - Cycle

The SOC cycle is a continuous process that ensures the effectiveness of security monitoring. It typically follows these phases:

- Preparation: Establishing the tools and team.
- Detection: Monitoring traffic to find anomalies.
- Analysis: Investigating the detected alerts to see if they are real threats.
- Response: Taking action to stop the threat (e.g., blocking an IP).
- Recovery & Lessons Learned: Restoring systems and improving the process based on the incident.

3.SIEM (Security Information and Event Management)

SIEM is the "brain" of the SOC. It is a technology that collects log data from various sources across the network (firewalls, servers, databases). We understood that SIEM provides two main functions: Information Management (storing logs) and Event Management (analyzing logs in real-time). It helps security analysts correlate different events to find a single complex attack.

4.SIEM Cycle

The SIEM cycle involves the flow of data to make it useful for analysts:

- Data Collection: Gathering logs from all devices.
- Data Normalization: Converting different log formats into a single standard format.
- Data Aggregation: Combining similar events to reduce noise.
- Correlation: Linking different events together using rules to identify an attack pattern.
- Alerting: Notifying the SOC team when a correlation rule is triggered.

5.MISP (Malware Information Sharing Platform)

MISP is an open-source threat intelligence platform. We understood that MISP allows organizations to share information about malware and cyber threats with each other. By using MISP, a SOC team can stay updated on the latest Indicators of Compromise (IoCs), such as malicious IP addresses or file hashes, which helps in proactive defense.

6.Your College Network Information

A typical college network is highly complex, consisting of thousands of users (students, faculty, and staff). It includes multiple Wi-Fi access points, computer labs, administrative servers, and library databases. Because of the open nature of a college network, it is highly susceptible to malware infections via student devices and phishing attacks.

7.How you think you deploy SOC in your college

To deploy a SOC in our college, I would follow a phased approach:

- Log Collection: Install agents on all department servers and core switches to send logs to a central SIEM.
- Tool Setup: Use an open-source or enterprise SIEM (like QRadar) to monitor the traffic.
- Student SOC Team: Form a team of trained students and faculty to monitor the dashboard during college hours.
- Incident Response: Create a policy where any detected threat leads to the automatic isolation of the affected lab or user account.

8.Threat Intelligence

Threat Intelligence is the knowledge of existing or emerging threats. It involves collecting data about attackers' motives and targets. In our project, we learned that integrating threat intelligence into a SIEM allows the SOC to know "who" is attacking and "how" they operate, rather than just seeing a random alert.

9.Incident Response

Incident Response (IR) is the organized approach to managing the aftermath of a security breach. The goal is to limit damage and reduce recovery time. We understood the "SANS" framework: Preparation, Identification, Containment, Eradication, Recovery, and Lessons Learned.

10. QRadar & Understanding about Tool

IBM QRadar is a powerful SIEM tool. We explored its dashboard, which provides a high-level view of "Offenses" (potential attacks). It uses "Magistrates" to prioritize alerts based on severity. We understood how QRadar analyzes network flows and log activity to provide a complete picture of the security environment.

Conclusion :-

Comprehensive Analysis of Security Operations

Stage 1: Insights from Web Application Testing

Through the initial phase of this project, I gained a profound understanding of the vulnerabilities that exist within the digital interface of an organization. Web application testing is not merely about finding bugs; it is about understanding the attacker's mindset. I learned how critical flaws like SQL Injection, Cross-Site Scripting (XSS), and Insecure Direct Object References (IDOR) can lead to catastrophic data breaches. The primary takeaway was that security must be integrated into the Software Development Life Cycle (SDLC) rather than being treated as an after thought.

Stage 2: Understanding the Nessus Vulnerability Report

The second stage shifted the focus from specific applications to broader network infrastructure. Analyzing a Nessus report taught me how to handle vast amounts of raw security data. I understood that a vulnerability scan is a diagnostic tool that identifies "open doors" such as outdated software, missing patches, and weak configurations. More importantly, I learned the art of Risk Prioritization—understanding that a "Critical" vulnerability on a core server requires immediate action compared to a "Medium" vulnerability on an isolated workstation. This stage bridged the gap between identifying a problem and planning a technical solution.

Stage 3: Mastery of SOC, SIEM, and QRadar Dashboards

The final stage provided a "bird’s-eye view" of enterprise security through the Security Operations Center (SOC) and SIEM tools like IBM QRadar. I understood that while Stages 1 and 2 are about finding holes, Stage 3 is about active monitoring and response. By exploring the QRadar dashboard, I learned how log correlation works—turning thousands of individual network events into a single "Offense" that an analyst can investigate. This stage taught me the importance of the SOC Cycle: from initial detection to the final lessons learned after an incident is closed.

Future Scope: The Next Frontier of Cybersecurity

1. AI and Machine Learning in Autonomous Defense

The future of Stage 3 (SOC/SIEM) is heavily transitioning toward Artificial Intelligence (AI) and Machine Learning (ML). Currently, security analysts have to manually write rules to detect threats. In the future, SIEM tools like QRadar will use "Self-Learning Algorithms" that understand the normal behavior of a network and automatically flag any tiny deviation without human intervention.

- Predictive Analytics: Instead of reacting to an attack that has already happened, AI will predict potential attack vectors based on global threat patterns.
- Automated Threat Hunting: AI bots will constantly "hunt" for hidden threats inside the network 24/7, reducing the "dwell time" (the time a hacker stays hidden in a system) from months to seconds.

2. Transition from SIEM to SOAR (The Automation Revolution)

While SIEM (Security Information and Event Management) is great for visibility, the future belongs to SOAR (Security Orchestration, Automation, and Response).

- Orchestration: This will allow different security tools (Firewalls, Email Security, Endpoint Protection) to "talk" to each other seamlessly.
- Playbook Automation: If a phishing email is detected, a SOAR playbook will automatically reset the user's password, scan their laptop, and block the malicious sender's IP across the entire company instantly. This removes the "human bottleneck" in incident response.

3. Zero Trust Architecture (ZTA)

The traditional idea of a "secure perimeter" (like a college firewall) is dying. The future scope involves moving toward a Zero Trust model.

- "Never Trust, Always Verify": In this future setup, even if you are inside the college Wi-Fi, the system will not trust your device automatically.
- Micro-segmentation: Each user and application will be isolated in its own tiny digital "bubble." Even if a hacker breaks into one student's laptop, they won't be able to move to the college's main server.

4. Quantum Cybersecurity and Post-Quantum Cryptography

As Quantum Computing becomes a reality, current encryption methods (like the ones used in Stage 1 Web Applications) will become easy to crack.

- Future Scope: Organizations will need to upgrade to Post-Quantum Cryptography (PQC). This involves developing new mathematical algorithms that even quantum computers cannot break.
- Stage 1 Impact: Web developers will have to rewrite how data is encrypted in transit and at rest to stay safe from quantum-level attacks.

5. DevSecOps: Security at the Speed of Code

In Stage 1 (Web Application Testing), we currently test apps after they are built. The future scope is DevSecOps, where security testing happens at every single second of the coding process.

- IAST (Interactive Application Security Testing): This is a future technology that monitors an application's inner workings while it's running to find vulnerabilities that traditional scanners miss.
- Security as Code: Security policies will be written as code, making them easy to update and deploy across thousands of cloud servers simultaneously.

6. IoT and 5G Security Challenges

With the rise of IoT (Internet of Things) and 5G, billions of new devices (smart cameras, sensors, medical devices) will connect to networks like your college network.

- Future SOC Responsibility: SOC teams will no longer just monitor laptops and servers. They will have to monitor thousands of "smart" devices that often have very weak built-in security.
- Edge Computing Security: Processing data closer to the user (Edge) will require a decentralized SOC model, where security is handled at the "edge" of the network rather than just in a central data center.

Topics Explored

In this comprehensive project, I explored several core domains of cybersecurity that are essential for modern digital defense:

1. **Vulnerability Management & Assessment:** I studied the systematic process of identifying, classifying, and prioritizing security weaknesses in an IT environment. This involved understanding how to differentiate between high-risk vulnerabilities that require immediate patching and low-risk flaws that can be monitored over time using the CVSS (Common Vulnerability Scoring System) framework.
2. **Log Management and Normalization:** I explored how massive amounts of raw data (logs) are collected from diverse sources like firewalls, routers, and servers. I understood the importance of "Normalization"—converting these different log formats into a single, readable standard so that a SIEM tool can analyze them effectively.
3. **Event Correlation and Rule Logic:** One of the most critical topics was learning how to connect the dots between seemingly unrelated network events. For example, I learned how a "failed login" followed by a "database access" from the same IP can trigger a correlation rule in a SIEM, identifying it as a potential brute-force attack.
4. **Threat Intelligence Integration:** I researched how external threat data (Global Threat Feeds) can be integrated into local security systems. This "Intelligence" allows a SOC team to be proactive—blocking malicious IPs and file hashes before they even interact with the organization's network.
5. **Incident Response Lifecycle (IR):** I studied the structured phases of responding to a security breach. Following the SANS/NIST framework, I explored the stages of **Preparation, Identification, Containment, Eradication, Recovery, and Lessons Learned**, ensuring a minimal impact on business continuity.

Tools Explored

Throughout the stages of this project, I gained hands-on familiarity with industry-standard security tools:

1. **IBM QRadar (SIEM Platform):** As a market-leading SIEM, I explored QRadar's ability to provide a centralized view of security. I learned how to navigate its dashboard, analyze "Offenses," and understand "Network Flows." It served as the primary tool for real-time monitoring and event correlation during Stage 3.

2. **Nessus (Vulnerability Scanner):** I utilized Nessus to perform automated network security audits. I explored how to configure scan policies, run credentialed vs. non-credentialed scans, and generate detailed reports that highlight missing patches and configuration errors across a network.
3. **Burp Suite (Web Vulnerability Scanner):** During the Web Application Testing phase (Stage 1), I used Burp Suite to intercept and analyze HTTP/HTTPS traffic. This tool was instrumental in understanding how attackers exploit web-level flaws like SQL injection and broken authentication.
4. **MISP (Malware Information Sharing Platform):** I explored MISP as a tool for managing Threat Intelligence. I learned how to store, share, and correlate Indicators of Compromise (IoCs) like malicious URLs and malware signatures, which are then used to feed information into the SIEM for better detection.

—----------THE END—----------